Base

AP4-1

https://172.26.17.11:8006/ *lien d'accès* proxmox

Réunion jour 1 :
https://docs.google.com/document/d/1qpwvd9Su7RcdhFZnaJ1ekOpkeABRpTVhpfsfpjJitcQ/edit?tab=t.0

Réunion jour 2 :
https://docs.google.com/document/d/1EQOixOX1skKT2ZHFh-hcoZJT3nSt0lRP_5Yy1l8tBd4/edit?tab=t.0

Réunion jour 3 :
https://docs.google.com/document/d/191hAKG5C4XoTSvVOY-sXPO3vtB8uxXI0lQzcdhLF7RQ/edit?usp=sharing

réunion jour 4 :
https://docs.google.com/document/d/1Ekqznc0eRexhyd1f0zgxx-_P5xMvaBRdd_uzyk6ryLM/edit?usp=sharing

réunion jour 5 :
https://docs.google.com/document/d/10Z8Yw6me8MmwKTWbzwu8G4avSIkYQFDHeuO2X59UVi4/edit?usp=sharing
Gestionnaire de serveur • DHCP
comment - Bloc-notes
Gérer Outils Affic
Fichier Edition Format Affichage Aide
le | <?xml version="1.0* encoding='utf-8°?>
oca
policyComments xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi-"http://www.w3.org/2
policyNamespaces>
kusing prefix-"ns0" namespace="Microsoft.Policies ControlAltDelete"›</using›
</policyNamespaces>
‹comments>
<admTemplate></admTemplate>
</comments>
<resources minRequiredRevision="1.0">
‹stringTable></stringTable>
</resources>
</policyComments>

Plan TCP/IP

Les 3 dernières de chaque vlans sont les 3 ip des routeurs.

	VLAN	De	à	Masque
Handball	10	172.26.11.1	172.26.11.59	255.255.255.192
Basket	20	172.26.11.65	172.26.11.91	255.255.255.224
Volley	30	172.26.11.97	172.26.11.123	255.255.255.224
Direction	80	172.26.11.129	172.26.11.155	255.255.255.224
Danse	40	172.26.11.161	172.26.11.171	255.255.255.240
Tennis	50	172.26.11.177	172.26.11.179	255.255.255.248
Escrime	60	172.26.11.185	172.26.11.187	255.255.255.248
Football	70	172.26.11.193	172.26.11.195	255.255.255.248
Bowling	90	172.26.11.201	172.26.11.203	255.255.255.248
DMZ	18	172.26.11.209	172.26.11.219	255.255.255.240
Wifi	100	172.26.11.225	172.26.11.235	255.255.255.240
Visiteurs	110	172.26.11.241	172.26.11.243	255.255.255.248
Téléphonie	300	172.26.13.1	172.26.13.123	255.255.255.128
Impression	15	172.26.15.1	172.26.15.11	255.255.255.240
Administration	99	172.26.16.1	172.26.16.11	255.255.255.240
serveur	17	172.26.17.1	172.26.17.11	255.255.255.240

	Passerelle Routeur 1	Passerelle Routeur 2	Passerelle Routeur VIP
Handball	172.26.11.60	172.26.11.61	172.26.11.62
Basket	172.26.11.92	172.26.11.93	172.26.11.94
Volley	172.26.11.124	172.26.11.125	172.26.11.126
Direction	172.26.11.156	172.26.11.157	172.26.11.158
Danse	172.26.11.172	172.26.11.173	172.26.11.174
Tennis	172.26.11.180	172.26.11.181	172.26.11.182
Escrime	172.26.11.188	172.26.11.189	172.26.11.190
Football	172.26.11.196	172.26.11.197	172.26.11.198
Bowling	172.26.11.204	172.26.11.205	172.26.11.206
DMZ	172.26.11.220	172.26.11.221	172.26.11.222
Wifi	172.26.11.236	172.26.11.237	172.26.11.238
Visiteurs	172.26.11.244	172.26.11.245	172.26.11.246
Téléphonie	172.26.13.124	172.26.13.125	172.26.13.126
Impression	172.26.15.12	172.26.15.13	172.26.15.14
Administration	172.26.16.12	172.26.16.13	172.26.16.14
serveur	172.26.17.12	172.26.17.13	172.26.17.14

Port

SWITCH COEUR ORBITE	
Interface	Affectation
GigabitEthernet1/0/1	LIEN ROUTEUR 1
GigabitEthernet1/0/2-3	LIEN SWITCH
GigabitEthernet1/0/4-6	VLAN 18 - DMZ
GigabitEthernet1/0/7-9	VLAN 17 - SERVEURS
GigabitEthernet1/0/10-16	VLAN 99 - ADMINISTRATION
GigabitEthernet1/0/17-24	VLAN 666 - SHUTDOWN

SWITCH ACCESS GALAXIE	
Interface	Affectation
GigabitEthernet1/0/1	LIEN ROUTEUR 2
GigabitEthernet1/0/2-3	LIEN SWITCH
GigabitEthernet1/0/4	VLAN 10 - HANDBALL
GigabitEthernet1/0/5	VLAN 20 – BASKET
GigabitEthernet1/0/6	VLAN 30 - VOLLEY
GigabitEthernet1/0/7	VLAN 80 - DIRECTION
GigabitEthernet1/0/8	VLAN 40 - DANSE
GigabitEthernet1/0/9	VLAN 50 - TENNIS
GigabitEthernet1/0/10	VLAN 60 - ESCRIME
GigabitEthernet1/0/11	VLAN 70 - FOOTBALL
GigabitEthernet1/0/12	VLAN 90 - BOWLING
GigabitEthernet1/0/13-14	VLAN 15 - IMPRIMANTE
GigabitEthernet1/0/15-16	VLAN 300 - TELEPHONIE
GigabitEthernet1/0/17-18	VLAN 110 - VISITEUR
GigabitEthernet1/0/19-24	VLAN 666 - SHUTDOWN

GLPI / SGBD

Mise en place d’une infrastructure GLPI haute disponibilité

1. Objectif du projet

L’objectif est de mettre en place une architecture **haute disponibilité** pour l’outil de gestion de parc informatique **GLPI**.

Cette architecture permet :

- la **redondance des serveurs web**
- la **synchronisation des bases de données**
- la **synchronisation des fichiers GLPI**
- la **continuité de service en cas de panne**

---

2. Architecture mise en place

L’infrastructure repose sur **4 machines virtuelles** :

Serveur	Rôle
GLPI1	serveur applicatif GLPI
GLPI2	serveur applicatif GLPI
SGBD1	serveur MariaDB principal
SGBD2	serveur MariaDB répliqué

Architecture :

```
      HAProxy (à venir)
     /                 \
   GLPI1   <-->   GLPI2
     |                 |
  SGBD1   <-->   SGBD2
```

Fonctionnement :

- GLPI1 utilise **SGBD1**
- GLPI2 utilise **SGBD2**
- les bases sont synchronisées via **réplication MariaDB**

---

3. Installation des serveurs GLPI

Installation des dépendances

Sur les serveurs GLPI :

```
apt update
apt install apache2 php php-mysql php-gd php-intl php-mbstring php-bz2 php-zip php-curl
php-ldap php-imap mariadb-client -y
```

Téléchargement de GLPI

```
cd /var/www/html

wget https://github.com/glpi-project/glpi/releases/download/11.0.5/glpi-11.0.5.tgz

tar -xvzf glpi-11.0.5.tgz
```

Permissions

```
chown -R www-data:www-data /var/www/html/glpi
chmod -R 755 /var/www/html/glpi
```

Configuration Apache

Création du VirtualHost :

```
nano /etc/apache2/sites-available/glpi.conf
```

Configuration :

```
<VirtualHost *:80>
```

```
DocumentRoot /var/www/html

Alias /glpi /var/www/html/glpi/public

<Directory /var/www/html/glpi/public>
  Options Indexes FollowSymLinks
  AllowOverride All
  Require all granted
</Directory>

</VirtualHost>
```

Activation :

```
a2ensite glpi.conf
a2enmod rewrite
systemctl reload apache2
```

Accès à GLPI :

http://IP_SERVEUR/glpi

---

4. Installation des serveurs MariaDB

Sur **SGBD1 et SGBD2** :

```
apt update
apt install mariadb-server -y
```

---

5. Configuration MariaDB pour la réplication

Fichier de configuration :

```
nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

Configuration SGBD1

```
[mysqld]
```

bind-address = 0.0.0.0

server-id = 1

log_bin = /var/log/mysql/mysql-bin.log

binlog_do_db = glpi

auto_increment_increment = 2
auto_increment_offset = 1

character-set-server = utf8mb4
collation-server = utf8mb4_general_ci

Configuration SGBD2

[mysqld]

bind-address = 0.0.0.0

server-id = 2

log_bin = /var/log/mysql/mysql-bin.log

binlog_do_db = glpi

auto_increment_increment = 2
auto_increment_offset = 2

character-set-server = utf8mb4
collation-server = utf8mb4_general_ci

Redémarrage MariaDB :

systemctl restart mariadb

---

6. Création de l’utilisateur de réplication

Sur les deux serveurs :

CREATE USER 'replication'@'%' IDENTIFIED BY 'replica123';

GRANT REPLICATION SLAVE ON *.* TO 'replication'@'%';

```
FLUSH PRIVILEGES;
```

---

7. Configuration de la réplication

Sur SGBD1 :

```
SHOW MASTER STATUS;
```

Exemple :

File: mysql-bin.000001
Position: 342

Configuration sur SGBD2 :

```
CHANGE MASTER TO
MASTER_HOST='IP_SGBD1',
MASTER_USER='replication',
MASTER_PASSWORD='replica123',
MASTER_LOG_FILE='mysql-bin.000001',
MASTER_LOG_POS=342;

START SLAVE;
```

La même opération est réalisée dans l'autre sens pour obtenir :

SGBD1 <--> SGBD2

---

8. Vérification de la réplication

Commande :

```
SHOW SLAVE STATUS\G
```

Résultat attendu :

Slave_IO_Running: Yes
Slave_SQL_Running: Yes

Test :

```
CREATE TABLE test_replication(
```

```
id INT PRIMARY KEY,
nom VARCHAR(50)
);

INSERT INTO test_replication VALUES (1,'cluster');
```

La table apparaît automatiquement sur **SGBD2**.

---

9. Migration de la base GLPI

Export de la base :

mysqldump -u root -p glpi > glpi.sql

Transfert :

```
scp glpi.sql root@IP_SGBD1:/root/
```

Création de la base :

```
CREATE DATABASE glpi;
```

Import :

mysql glpi < glpi.sql

La base est automatiquement synchronisée vers **SGBD2**.

---

10. Configuration de GLPI

Fichier :

```
/var/www/html/glpi/config/config_db.php
```

GLPI1

```
$dbhost = 'IP_SGBD1';
$dbuser = 'glpi';
$dbpassword = 'motdepasse';
$dbdefault = 'glpi';
```

GLPI2

```
$dbhost = 'IP_SGBD2';
$dbuser = 'glpi';
$dbpassword = 'motdepasse';
$dbdefault = 'glpi';
```

---

11. Synchronisation des fichiers GLPI

GLPI stocke les fichiers dans :

/var/www/html/glpi/files

Ces fichiers doivent être synchronisés entre les deux serveurs.

Installation rsync

```
apt install rsync -y
```

Synchronisation manuelle

GLPI1 → GLPI2

```
rsync -az /var/www/html/glpi/files/ root@GLPI2:/var/www/html/glpi/files/
```

GLPI2 → GLPI1

```
rsync -az /var/www/html/glpi/files/ root@GLPI1:/var/www/html/glpi/files/
```

Synchronisation automatique

GLPI1 :

```
crontab -e
*/1 * * * * /usr/bin/rsync -az /var/www/html/glpi/files/ root@GLPI2:/var/www/html/glpi/files/
```

GLPI2 :

```
*/1 * * * * /usr/bin/rsync -az /var/www/html/glpi/files/ root@GLPI1:/var/www/html/glpi/files/
```

La synchronisation est effectuée **toutes les minutes**.

---

12. Tests réalisés

Tests effectués :

Synchronisation des bases

Création d'un ticket sur GLPI1.

Résultat :

- le ticket apparaît sur GLPI2
- confirmation du bon fonctionnement de la réplication MariaDB

Synchronisation des fichiers

Création d'un fichier dans :

/var/www/html/glpi/files

Résultat :

- le fichier apparaît sur l'autre serveur après synchronisation rsync

---

13. Résultat obtenu

L'infrastructure fonctionne maintenant comme suit :

```
 GLPI1   <-->   GLPI2
    |                |
    |                |
SGBD1  <-->  SGBD2
```

Les éléments synchronisés :

- base de données GLPI

- fichiers GLPI
- configuration applicative

---

14. Étape suivante

La prochaine étape consiste à mettre en place un **load balancer HAProxy** afin de répartir les connexions utilisateurs entre les deux serveurs GLPI.

Architecture finale :

```
        HAProxy
      /          \
  GLPI1  <-->  GLPI2
    |              |
 SGBD1 <--> SGBD2
```

HAProxy + HTTPS

Documentation – Mise en place HAProxy avec GLPI en haute disponibilité

1️⃣ Objectif

Mettre en place une architecture avec :

- 2 serveurs GLPI (GLPI1 / GLPI2)
- 1 répartiteur de charge HAProxy
- Accès sécurisé en HTTPS
- Séparation réseau (critère BTS)

🧱 Architecture finale

CLIENT (172.26.11.x)

↓

HAProxy (172.26.11.5) ← ens18 (réseau client)

↓

HAProxy (172.26.17.4) ← ens19 (réseau serveur)

↓

GLPI1 (172.26.17.2)

GLPI2 (172.26.17.3)

2️⃣ Configuration réseau HAProxy

📄 Fichier : `/etc/network/interfaces`

```
auto lo
iface lo inet loopback

# Réseau CLIENTS
auto ens18
iface ens18 inet static
  address 172.26.11.5
  netmask 255.255.255.240
  gateway 172.26.11.14

# Réseau SERVEURS
auto ens19
iface ens19 inet static
  address 172.26.17.4
  netmask 255.255.255.240
```

🔁 Application

```
systemctl restart networking
# ou
reboot
```

✅ Vérification

```
ip a
```

Résultat attendu :

- ens18 → 172.26.11.5
- ens19 → 172.26.17.4

3️⃣ Configuration HAProxy

📄 Fichier : `/etc/haproxy/haproxy.cfg`

```
frontend glpi_front_http
  bind 172.26.11.5:80
  mode http
  redirect scheme https code 301 if !{ ssl_fc }

frontend glpi_front_https
  bind 172.26.11.5:443 ssl crt /etc/haproxy/certs/glpi.pem
  mode http
  option forwardfor
  http-request set-header X-Forwarded-Proto https
  http-request set-header X-Forwarded-Port 443
  default_backend glpi_servers

backend glpi_servers
  mode http
  balance roundrobin
  option httpchk

  cookie SERVERID insert indirect nocache
```

```
    server glpi1 172.26.17.2:80 check cookie s1
    server glpi2 172.26.17.3:80 check cookie s2

listen stats
    bind 172.26.11.5:8404
    mode http
    stats enable
    stats uri /stats
```

🔁 Redémarrage

```
haproxy -c -f /etc/haproxy/haproxy.cfg
systemctl restart haproxy
```

4️⃣ Problème rencontré (important pour le recettage)

❌ Erreur 503

Cause :

HAProxy ne voyait aucun serveur disponible

Solution :

passage en option tcp-check

❌ Layer7 timeout

Cause :

GLPI trop lent pour répondre au health check HTTP

❌ SSL handshake failed

Cause :

certificat incorrect (IP différente)

5️⃣ Génération du certificat HTTPS

📄 Création du fichier OpenSSL

nano /root/openssl.cnf

```
[req]

distinguished_name = req_distinguished_name

x509_extensions = v3_req

prompt = no


[req_distinguished_name]

CN = 172.26.11.5


[v3_req]

subjectAltName = @alt_names


[alt_names]

IP.1 = 172.26.11.5
```

🔐 Génération du certificat

```
openssl req -x509 -nodes -days 365 \
-newkey rsa:2048 \
-keyout /etc/haproxy/certs/glpi.key \
-out /etc/haproxy/certs/glpi.crt \
-config /root/openssl.cnf
```

📦 Création du fichier `.pem`

```
cat /etc/haproxy/certs/glpi.key /etc/haproxy/certs/glpi.crt > /etc/haproxy/certs/glpi.pem
```

🔁 Redémarrage HAProxy

```
systemctl restart haproxy
```

6️⃣ Installation du certificat sur le client

📥 Export

```
scp /etc/haproxy/certs/glpi.crt user@pc:/destination
```

🪟 Installation Windows

1. Ouvrir le fichier `.crt`
2. Installer le certificat
3. Choisir :
    - Ordinateur local
    - Autorités de certification racines de confiance

🔄 Redémarrer navigateur

7️⃣ Tests

🌐 Accès

http://172.26.11.5

https://glpi.m2l1.lan

📊 Statistiques HAProxy

http://172.26.11.5:8404/stats

🧪 Test de load balancing

- Rafraîchir la page
- Vérifier alternance GLPI1 / GLPI2

8️⃣ Validation des critères BTS

✔ Haute disponibilité

- 2 serveurs GLPI
- répartition via HAProxy

✔ Sécurisation HTTPS

- certificat SSL
- redirection HTTP → HTTPS

✔ Séparation réseau

- ens18 → réseau client
- ens19 → réseau serveur

✔ Architecture professionnelle

- reverse proxy
- backend isolé
- accès unique

🎯 Conclusion

L'infrastructure mise en place permet :

- une haute disponibilité de l'application GLPI
- une sécurisation des communications via HTTPS
- une séparation logique des réseaux
- une architecture conforme aux bonnes pratiques professionnelles

Active Directory / LDAPS

Documentation — Implémentation de l'authentification LDAPS entre GLPI et Active Directory

1. Objectif

L'objectif de cette configuration est de permettre à **GLPI** d'utiliser l'**authentification centralisée Active Directory** pour les utilisateurs du système d'information.

L'authentification est sécurisée grâce au protocole **LDAPS (LDAP Secure)** utilisant le **port 636** et le chiffrement **TLS**.

Les avantages sont :

- centralisation de la gestion des comptes utilisateurs
- authentification sécurisée
- suppression de la gestion locale des mots de passe dans GLPI
- intégration avec l'infrastructure Active Directory existante

---

2. Architecture

Infrastructure utilisée

Machine	Rôle	Adresse IP
GLPI1	Serveur GLPI	192.168.1.25
WIN-AD	Contrôleur de domaine Active Directory	192.168.1.30

Domaine Active Directory

glpi.local

Organisation des utilisateurs

OU=GLPI_Users

Schéma d'architecture

```
Utilisateurs AD
      │
      │ Authentification
      ▼
GLPI (192.168.1.25)
      │
      │ LDAPS (port 636)
      ▼
Active Directory (192.168.1.30)
```

---

3. Installation de GLPI

Sur le serveur Linux (Debian/Ubuntu).

Mise à jour du système

```
apt update
apt upgrade -y
```

Installation du serveur web

```
apt install apache2
```

Installation de PHP et modules nécessaires

```
apt install php php-cli php-common php-curl php-gd php-intl php-mysql php-xml
php-mbstring php-bcmath php-zip php-ldap
```

Le module **php-ldap** est nécessaire pour permettre à GLPI de communiquer avec Active Directory.

Installation de MariaDB

```
apt install mariadb-server
```

Création de la base GLPI

Connexion à MariaDB :

mysql -u root -p

Création de la base :

```
CREATE DATABASE glpi;
CREATE USER 'glpi'@'localhost' IDENTIFIED BY 'motdepasse';
GRANT ALL PRIVILEGES ON glpi.* TO 'glpi'@'localhost';
FLUSH PRIVILEGES;
```

Installation de GLPI

Téléchargement :

```
wget https://github.com/glpi-project/glpi/releases/download/10.x/glpi-10.x.tgz
```

Extraction :

```
tar -xvzf glpi-10.x.tgz
```

Déplacement dans le serveur web :

```
mv glpi /var/www/html/
```

Permissions :

```
chown -R www-data:www-data /var/www/html/glpi
```

Accès à l'installation

Dans un navigateur :

http://192.168.1.25/glpi

Suivre l'assistant d'installation.

---

4. Vérification du support LDAP

Sur le serveur GLPI :

```
php -m | grep ldap
```

Résultat attendu :

```
ldap
```

---

5. Vérification de la connectivité LDAPS

Test de la connexion SSL avec le contrôleur de domaine.

```
openssl s_client -connect 192.168.1.30:636 -showcerts
```

Résultat attendu :

```
CONNECTED
Verification: OK
TLSv1.2
```

Cela confirme :

- le port **636 est accessible**
- le certificat **Active Directory est valide**

---

6. Test LDAP depuis le serveur GLPI

Installation des outils LDAP :

```
apt install ldap-utils
```

Test de recherche LDAP :

```
ldapsearch -x \
-H ldaps://192.168.1.30 \
-D "test.glpi@glpi.local" \
-W \
-b "DC=glpi,DC=local"
```

Résultat attendu :

```
result: 0 Success
```

Cela confirme que :

- l'authentification LDAP fonctionne
- les objets Active Directory sont accessibles

---

7. Configuration LDAP dans GLPI

Dans l'interface GLPI :

Configuration
→ Authentification
→ LDAP
→ Ajouter

Paramètres du serveur LDAP

Nom :

Active Directory

Serveur :

ldaps://WIN-AD.glpi.local

Port :

636

Base DN

OU=GLPI_Users,DC=glpi,DC=local

Compte de connexion

test.glpi@glpi.local

Mot de passe du compte AD.

Filtre utilisateur

(&(objectCategory=person)(objectClass=user))

Champ identifiant

sAMAccountName

Champ de synchronisation

objectGUID

---

8. Test de la configuration LDAP

Dans GLPI :

Tester

Résultat attendu :

Connexion réussie
Recherche réussie

---

9. Import des utilisateurs Active Directory

Dans GLPI :

Administration
→ Utilisateurs
→ Importer depuis un annuaire LDAP

Les utilisateurs présents dans :

OU=GLPI_Users

apparaissent dans GLPI.

---

10. Attribution des profils

Chaque utilisateur doit recevoir un profil.

Exemple :

Utilisateur	Profil
utilisateurs	Self-Service
support IT	Technicien
administrateur	Super-Admin

---

11. Vérification réseau

Sur le contrôleur de domaine :

netstat -an | findstr 636

Résultat attendu :

TCP 0.0.0.0:636 LISTENING

Cela confirme que **LDAPS est actif sur Active Directory**.

---

12. Résultat final

Les utilisateurs Active Directory peuvent maintenant :

- se connecter à GLPI avec leurs identifiants AD
- être authentifiés via **LDAPS sécurisé**
- utiliser les fonctionnalités GLPI selon leur profil

L'authentification centralisée permet :

- une gestion simplifiée des comptes
- une meilleure sécurité
- une intégration complète avec l'infrastructure Microsoft.

Agent GLPI

http://172.26.17.3/glpi/front/inventory.php, http://172.26.17.2/glpi/front/inventory.php

Snort/Splunk / Fail2ban / OpenVas

Documentation — Mise en place et vérification de Fail2Ban (Debian) — VM “Fail2Ban” (local + centralisé)

Sommaire

1. Objectif
2. Pré-requis
3. Installation de Fail2Ban
4. Préparer les logs centralisés (`/var/log/remote/...`)
5. Principe de configuration Fail2Ban (bonnes pratiques)
6. Activer la jail SSH locale `sshd` (log local)
7. Créer la jail SSH centralisée `sshd-central` (logs distants)
8. Créer la jail HAProxy centralisée `haproxy-central` (log distant)
9. Filtres (regex) : comment valider sans inventer
10. Redémarrage et rechargement
11. Vérifications finales (comme sur ta VM)
12. Tests (preuve que “ça fonctionne”)
13. À retenir (checklist rapide)
14. Sauvegarde de la configuration (optionnel)

---

1) Objectif

Refaire une VM Fail2Ban Debian qui protège :

- le SSH local via la jail `sshd` et le log `/var/log/auth.log`
- le SSH centralisé via la jail `sshd-central` et les logs :
  - `/var/log/remote/HAProxy/sshd.log`
  - `/var/log/remote/SGBD1/sshd.log`
  - `/var/log/remote/Fail2Ban/sshd.log`
- HAProxy centralisé via la jail `haproxy-central` et le log :
  - `/var/log/remote/HAProxy/haproxy.log`

Les valeurs exactes (durées, seuils, action firewall, ignoreip) ne sont pas fixées ici (tu as demandé de ne pas fournir plus), mais la procédure est complète et reproductible.

---

2) Pré-requis

- Debian avec accès `root`
- Logs SSH disponibles localement dans `/var/log/auth.log`
- Les logs centralisés arrivent (d'une manière ou d'une autre) dans `/var/log/remote/...`
  (rsyslog, syslog-ng, copie, agent… — non détaillé ici)

---

3) Installation de Fail2Ban

bash

```bash
apt update
apt install -y fail2ban
systemctl enable --now fail2ban
```

Vérification :

bash

```bash
systemctl status fail2ban --no-pager -l
fail2ban-client ping
```

---

4) Préparer les logs centralisés (`/var/log/remote/...`)

Créer l'arborescence :

bash

```bash
mkdir -p /var/log/remote/HAProxy
mkdir -p /var/log/remote/SGBD1
mkdir -p /var/log/remote/Fail2Ban
```

Créer les fichiers s'ils n'existent pas (utile pour que Fail2Ban puisse les ouvrir) :

```bash
touch /var/log/remote/HAProxy/sshd.log
touch /var/log/remote/SGBD1/sshd.log
touch /var/log/remote/Fail2Ban/sshd.log
touch /var/log/remote/HAProxy/haproxy.log
```

Permissions de base :

```bash
chown -R root:adm /var/log/remote 2>/dev/null || chown -R root:root
/var/log/remote
chmod -R 750 /var/log/remote
chmod 640 /var/log/remote/HAProxy/*.log /var/log/remote/SGBD1/*.log
/var/log/remote/Fail2Ban/*.log 2>/dev/null || true
```

---

5) Principe de configuration Fail2Ban (bonnes pratiques)

- Ne pas modifier `/etc/fail2ban/jail.conf`
- Mettre ta config dans :
  - `/etc/fail2ban/jail.d/*.local` (recommandé)
  - ou `/etc/fail2ban/jail.local`

---

6) Activer la jail SSH locale `sshd` (log local)

Créer :

```bash
nano /etc/fail2ban/jail.d/sshd.local
```

Contenu minimal (tes chemins) :

```INI
[sshd]
enabled = true
```

```
logpath = /var/log/auth.log
```

---

7) Créer la jail SSH centralisée `sshd-central` (logs distants)

Créer :

```bash
nano /etc/fail2ban/jail.d/sshd-central.local
```

Contenu (tes chemins EXACTS) :

```INI
[sshd-central]
enabled = true
logpath = /var/log/remote/HAProxy/sshd.log
          /var/log/remote/SGBD1/sshd.log
          /var/log/remote/Fail2Ban/sshd.log
```

---

8) Créer la jail HAProxy centralisée `haproxy-central` (log distant)

Créer :

```bash
nano /etc/fail2ban/jail.d/haproxy-central.local
```

Contenu (tes chemins EXACTS) :

```INI
[haproxy-central]
enabled = true
logpath = /var/log/remote/HAProxy/haproxy.log
```

---

9) Filtres (regex) : comment valider sans inventer

Fail2Ban a besoin de filtres (regex) pour compter des “failed”.

9.1 SSH

Le filtre standard SSH est généralement déjà présent :

- `/etc/fail2ban/filter.d/sshd.conf`

Tu peux vérifier que des lignes matchent :

```bash
fail2ban-regex /var/log/auth.log /etc/fail2ban/filter.d/sshd.conf
fail2ban-regex /var/log/remote/HAProxy/sshd.log
/etc/fail2ban/filter.d/sshd.conf
```

9.2 HAProxy

Le filtre dépend de ton format de log HAProxy et de ce que tu veux bannir. Méthode recommandée :

1. choisir/créer un filtre dans `/etc/fail2ban/filter.d/`
2. tester avec :

```bash
fail2ban-regex /var/log/remote/HAProxy/haproxy.log
/etc/fail2ban/filter.d/<ton-filtre-haproxy>.conf
```

---

10) Redémarrage et rechargement

Après modifications :

```bash
systemctl restart fail2ban
```

---

11) Vérifications finales (comme sur ta VM)

11.1 Confirmer que les 3 jails sont bien chargées

bash

```bash
fail2ban-client status
```

Attendu :

- Number of jail: 3
- Jail list: `haproxy-central, sshd, sshd-central`

11.2 Confirmer les logpath par jail

bash

```bash
fail2ban-client status sshd
fail2ban-client status sshd-central
fail2ban-client status haproxy-central
```

Attendu (points essentiels) :

- `sshd` → `File list: /var/log/auth.log`
- `sshd-central` → `File list:` avec les 3 fichiers `/var/log/remote/.../sshd.log`
- `haproxy-central` → `File list: /var/log/remote/HAProxy/haproxy.log`

11.3 Logs Fail2Ban (diagnostic)

bash

```bash
tail -n 200 /var/log/fail2ban.log
```

---

12) Tests (preuve que “ça fonctionne”)

12.1 Test rapide d’action (ban/unban manuel)

Permet de valider l'action sans attendre des échecs :

```bash
fail2ban-client set sshd-central banip 1.2.3.4
fail2ban-client status sshd-central
fail2ban-client set sshd-central unbanip 1.2.3.4
```

12.2 Test réel via logs

- Générer des échecs SSH (local ou distants) pour faire augmenter `Total failed`.
- Vérifier ensuite :

```bash
fail2ban-client status sshd
fail2ban-client status sshd-central
```

---

13) À retenir (checklist rapide)

1. Installer Fail2Ban + démarrer le service
2. Créer `/var/log/remote/{HAProxy,SGBD1,Fail2Ban}`
3. Activer 3 jails et mettre les `logpath` :
   - `sshd` → `/var/log/auth.log`
   - `sshd-central` → 3 logs distants
   - `haproxy-central` → log HAProxy distant
4. Redémarrer Fail2Ban
5. Vérifier avec `fail2ban-client status` + `fail2ban-client status <jail>`

---

14) Sauvegarde de la configuration (optionnel)

```bash
tar -czf /root/fail2ban-config-backup.tar.gz /etc/fail2ban
```

SNORT/SPLUNK

0) Pré-check côté VM (réseau + temps + espace)

0.1 Trouver l'IP de la VM

```bash
ip -br addr
```

Pourquoi : tu en auras besoin pour :

- envoyer le fichier depuis ton PC (`scp`)
- accéder à l'UI Splunk (`http://IP_VM:8000`)

0.2 Vérifier l'espace disque (Splunk consomme vite)

```bash
df -h /
df -h /opt || true
```

Pourquoi : Splunk stocke ses données sous `/opt/splunk/var`. Il faut de la place (plusieurs Go minimum).

0.3 Vérifier l'heure (important pour corrélation de logs)

```bash
date
timedatectl status
```

---

1) Télécharger Splunk Enterprise sur ton PC

1. Va sur le site Splunk et télécharge Splunk Enterprise for Linux (.deb, x86_64).
2. Tu obtiens un fichier du style :

- `splunk-9.x.x-<build>-linux-2.6-amd64.deb`

Pourquoi : Splunk n'est pas dans les dépôts Debian officiels ; l'installation standard se fait via paquet fourni par Splunk.

---

2) Envoyer le .deb depuis ton PC vers la VM

2.1 (Recommandé) Envoyer via `scp`

Sur ton PC (macOS/Linux/WSL), dans le dossier où se trouve le `.deb` :

```bash
scp splunk-*.deb root@IP_VM:/tmp/
```

Pourquoi : `/tmp` est un endroit simple pour déposer un paquet avant installation.

Si tu n'as pas accès root en SSH

Envoie vers un user (ex: `debian`) :

```bash
scp splunk-*.deb debian@IP_VM:/tmp/
```

Puis tu feras `sudo` sur la VM.

2.2 Vérifier côté VM que le fichier est bien arrivé

Sur la VM :

```bash
ls -lh /tmp/splunk-*.deb
```

---

3) Installer Splunk Enterprise sur Debian 12 (sur la VM)

3.1 Installer le paquet .deb

Sur la VM :

```bash
apt update
apt install -y /tmp/splunk-*.deb
```

Pourquoi :

- `apt install ./fichier.deb` (ou `/tmp/...`) gère les dépendances proprement.
- Splunk s'installe normalement dans `/opt/splunk`.

3.2 Vérifier que Splunk est bien installé

```bash
ls -la /opt/splunk
/opt/splunk/bin/splunk version
```

---

4) Premier démarrage + création du compte admin

4.1 Démarrer Splunk et accepter la licence

```bash
/opt/splunk/bin/splunk start --accept-license --answer-yes --no-prompt
```

Pourquoi :

- Premier démarrage = acceptation licence obligatoire.
- Splunk va te demander de définir un compte admin (login/mot de passe).
- Si l'assistant te le demande en interactif, suis les instructions.

Si tu veux forcer user/pass admin en CLI (option avancée)

Dis-moi si tu veux cette méthode, je te donne la commande exacte (ça dépend des options et de la version).

4.2 Vérifier que le port web écoute (8000)

```bash
ss -lntp | grep -E ':8000\b' || true
```

Pourquoi : confirmer que Splunk Web est disponible.

---

5) Ouvrir l'UI Splunk depuis ton PC

Dans ton navigateur :

- `http://IP_VM:8000`

Login :

- utilisateur : `admin` (souvent)
- mot de passe : celui que tu as défini au 1er start

---

6) Activer Splunk au démarrage (boot-start)

Splunk propose une commande officielle pour générer/configurer le service :

6.1 (Simple) Activer boot-start en root (rapide)

```bash
/opt/splunk/bin/splunk enable boot-start
```

Ensuite :

```bash
systemctl status splunk --no-pager
systemctl enable --now splunk
```

Pourquoi : tu veux que Splunk redémarre automatiquement après reboot.

Bonnes pratiques : faire tourner Splunk avec un user dédié (`splunk`) plutôt que root. Si tu veux la version “propre/sécure”, je te la fais (création user, droits /opt/splunk, boot-start -user splunk).

---

7) Pare-feu (si besoin)

Si tu utilises `ufw` :

```bash
apt install -y ufw
ufw allow 8000/tcp
ufw status
```

Pourquoi : autoriser l’accès à Splunk Web.

(Si tu n’as pas de firewall actif, cette étape ne change rien.)

---

8) Test ingestion de logs (pour valider que Splunk indexe)

8.1 Ajouter un “monitor” sur /var/log (simple)

```bash
/opt/splunk/bin/splunk add monitor /var/log
```

Pourquoi : tu verras immédiatement des événements Linux dans Splunk.

8.2 Requête de test dans l’UI

Dans “Search & Reporting”, teste :

- `index=_internal | head 20`
- `source="/var/log/auth.log" | head 50`

---

9) Commandes utiles (opérations courantes)

Démarrer / arrêter / redémarrer :

```bash
/opt/splunk/bin/splunk start
/opt/splunk/bin/splunk stop
/opt/splunk/bin/splunk restart
```

Logs Splunk (diagnostic) :

```bash
ls -la /opt/splunk/var/log/splunk
```

```
tail -n 200 /opt/splunk/var/log/splunk/splunkd.log
```

OPENVAS

0) Pré-requis (VM Linux)

Recommandé minimum (lab) :

- 4 vCPU (8 mieux)
- 8 Go RAM (16 mieux)
- 60–100 Go disque (feeds + DB + reports)
- Accès Internet sortant (rsync/https) + DNS OK

Paquets :

```bash
sudo apt update
sudo apt install -y docker.io docker-compose-plugin curl
sudo systemctl enable --now docker
```

---

1) Installation avec persistance (recommandé)

Le point clé : utiliser des volumes pour ne pas retélécharger/importer les feeds à chaque reboot.

1.1 Créer les volumes

```bash
docker volume create openvas-data
docker volume create openvas-var-lib-openvas
docker volume create openvas-var-lib-gvm
```

```
docker volume create openvas-var-lib-notus
```

1.2 Lancer le conteneur

Remplace l'image si tu utilises une autre (mais l'idée reste identique).

bash

```
docker run -d \
  --name openvas \
  --restart unless-stopped \
  -p 9392:9392 \
  -v openvas-data:/data \
  -v openvas-var-lib-openvas:/var/lib/openvas \
  -v openvas-var-lib-gvm:/var/lib/gvm \
  -v openvas-var-lib-notus:/var/lib/notus \
  immauss/openvas:latest
```

Vérifier

bash

```
docker ps
docker logs -f openvas
```

---

2) Premier démarrage (feeds) : quoi attendre

Au premier lancement (ou si pas persistant), ça fait :

- démarrage redis/postgres
- sync feeds (NASL/Notus/SCAP/CERT)
- import en DB

Ça peut prendre 10–60 minutes selon ta VM + Internet.

2.1 Attendre l'état healthy

```bash
watch -n 2 'docker inspect -f "{{.State.Health.Status}}" openvas'
```

Quand ça devient `healthy`, l'UI est prête.

---

3) Accès Web (GSA)

Dans ton navigateur :

- HTTP : `http://IP_DE_TA_VM:9392`

Important : si tu testes avec `https://`, tu auras des erreurs SSL. Par défaut c'est du HTTP sur 9392 (dans cette image).

---

4) Création / reset du compte admin

Selon les images, `admin` peut exister ou non. La méthode fiable :

4.1 Vérifier les users (DB)

```bash
docker exec -it openvas sh -lc "psql -U postgres -d gvmd -c 'select name from users order by name;'"
```

4.2 Reset mot de passe admin (si admin existe)

(évite `su - gvm` si HOME absent, utilise `su gvm`)

```bash
docker exec -it openvas sh -lc 'su gvm -c "gvmd --user=admin --new-password=Azerty11"'
```

Ensuite login UI :

- user: `admin`
- pass: `Azerty11`

---

5) Vérifications “santé” (à connaître)

5.1 Scanner présent

```bash
docker exec -it openvas sh -lc 'su gvm -c "gvmd --get-scanners"'
```

Tu dois voir OpenVAS Default et le socket `/run/ospd/ospd-openvas.sock`.

5.2 Socket OSPD

```bash
docker exec -it openvas sh -lc 'ls -la /var/run/ospd/ospd-openvas.sock'
```

---

6) Créer un scan (UI)

6.1 Target

Configuration → Targets → New Target

- Name: `target-vm`
- Hosts: `172.26.17.9` (1 IP pour commencer)
- Port list: par défaut
- Alive test: “ICMP, TCP-ACK …” (LAN)

6.2 Task

Scans → Tasks → New Task

- Name: `scan-target-vm`
- Scan Config: `Full and fast` (début)
- Scanner: `OpenVAS Default`
- Target: `target-vm`

Puis clique Start.

Si tu vois “Feed is currently syncing”, attends que le conteneur soit `healthy` et recharge (Ctrl+F5).

Docker GLPI

Installation de Docker et déploiement de GLPI avec transfert vers une autre VM

1. Objectif

Mettre en place GLPI dans un conteneur Docker sur une première VM, puis transférer la configuration vers une seconde VM afin de reproduire l'installation.

---

2. Installation de Docker sur la VM GLPI

2.1 Mise à jour du système

apt update && apt upgrade -y

2.2 Installation de Docker et Docker Compose

apt install -y docker.io docker-compose

2.3 Activation du service Docker

systemctl enable --now docker

2.4 Vérification

docker --version
docker ps

---

3. Création du projet GLPI avec Docker

3.1 Création du dossier

mkdir /opt/glpi-docker
cd /opt/glpi-docker

3.2 Création du fichier docker-compose.yml

nano docker-compose.yml

3.3 Contenu du fichier

version: '3.8'

services:

```
  db:
    image: mariadb:10.6
    container_name: glpi-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: glpi
      MYSQL_USER: glpi
      MYSQL_PASSWORD: glpi
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi
    container_name: glpi-app
    restart: always
    ports:
      - "8080:80"
    depends_on:
      - db
    environment:
      TIMEZONE: Europe/Paris
    volumes:
      - glpi_data:/var/www/html/glpi

volumes:
  db_data:
  glpi_data:
```

---

4. Lancement de GLPI

docker-compose up -d

Vérification

docker ps

Accès

http://IP_VM:8080

---

5. Transfert du projet vers une autre VM (SCP)

5.1 Principe

Le transfert permet de copier la configuration Docker afin de reproduire l'installation sur une autre machine.

5.2 Commande SCP (depuis la VM source)

scp -r /opt/glpi-docker root@IP_VM2:/opt/

Remplacer :

- `IP_VM2` par l'adresse IP de la seconde VM

5.3 Vérification sur la VM cible

ls /opt/

Le dossier `glpi-docker` doit être présent.

---

6. Déploiement sur la seconde VM

6.1 Accès au dossier

cd /opt/glpi-docker

6.2 Lancement des conteneurs

docker-compose up -d

---

7. Résultat

- VM1 : GLPI accessible sur `http://IP_VM1:8080`
- VM2 : GLPI accessible sur `http://IP_VM2:8080`

Chaque instance est indépendante mais basée sur la même configuration Docker.

---

8. Conclusion

Cette méthode permet :

- une installation rapide et reproductible de GLPI
- un déploiement sur plusieurs VM

- une base pour la mise en place d’un cluster (avec base de données centralisée et load balancer)

Docker SGBD

Installation de Docker et déploiement d’un SGBD MariaDB avec transfert vers une autre VM

---

1. Objectif

Mettre en place un serveur de base de données MariaDB dans un conteneur Docker sur une première machine virtuelle (VM SGBD), puis transférer cette configuration vers une seconde VM afin de reproduire rapidement l’installation.

---

2. Installation de Docker sur la VM SGBD

2.1 Mise à jour du système

```
apt update && apt upgrade -y
```

2.2 Installation de Docker et Docker Compose

```
apt install -y docker.io docker-compose
```

2.3 Activation du service Docker

```
systemctl enable --now docker
```

2.4 Vérification

```
docker --version
```

---

3. Création de l’environnement Docker

3.1 Création du dossier

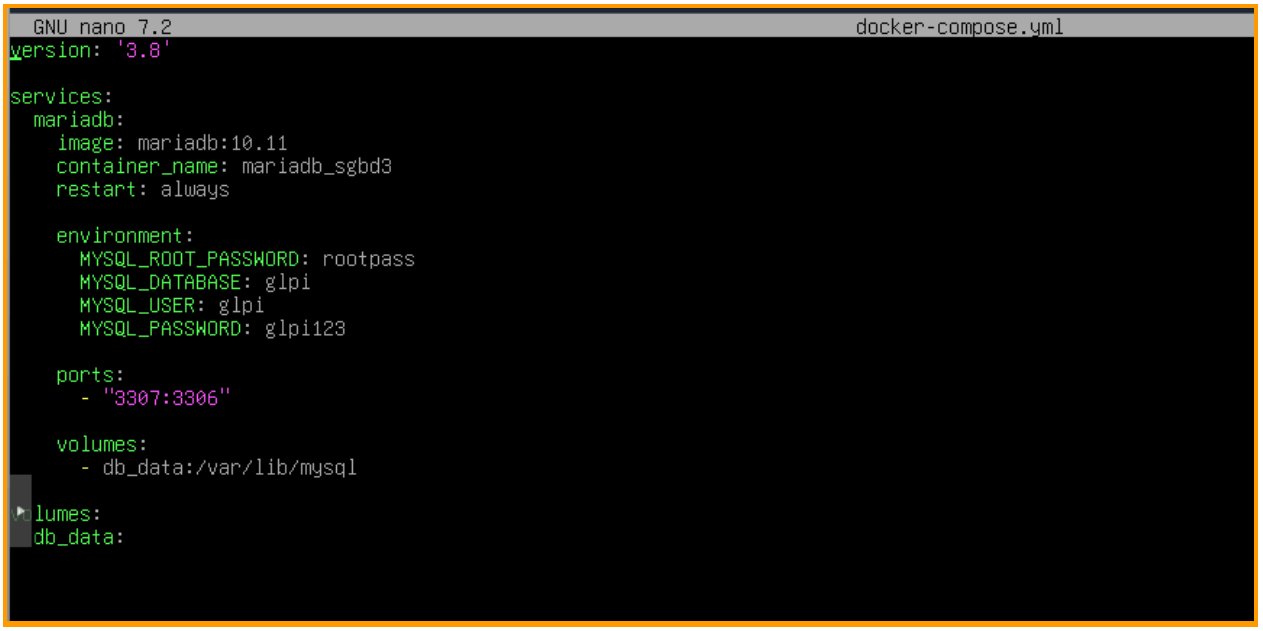
```
mkdir /opt/mariadb-docker
cd /opt/mariadb-docker
```

3.2 Création du fichier docker-compose

```
nano docker-compose.yml
```

Contenu du fichier :

```
version: '3.8'

services:
 mariadb:
  image: mariadb:10.11
  container_name: mariadb_sgbd
  restart: always

  environment:
    MYSQL_ROOT_PASSWORD: rootpass
    MYSQL_DATABASE: glpi
    MYSQL_USER: glpi
    MYSQL_PASSWORD: glpi123

  ports:
    - "3307:3306"

  volumes:
    - db_data:/var/lib/mysql

volumes:
 db_data:
```



---

4. Déploiement du SGBD

4.1 Lancement du conteneur

docker-compose up -d

4.2 Vérification du conteneur

```
docker ps
```

---

5. Test du fonctionnement

5.1 Connexion au SGBD

```
mysql -h 127.0.0.1 -P 3307 -u glpi -p
```

5.2 Résultat attendu

Connexion au serveur MariaDB avec affichage du terminal SQL.

---

6. Transfert de la configuration vers une autre VM

6.1 Copie du dossier via SCP

Depuis la VM source :

```
scp -r /opt/mariadb-docker root@IP_SGBDX:/opt/
```

---

7. Déploiement sur la VM destination

7.1 Connexion à la VM

```
ssh root@IP_SGBDX
```

7.2 Accès au dossier

```
cd /opt/mariadb-docker
```

7.3 Installation de Docker

```
apt update && apt install -y docker.io docker-compose
systemctl enable --now docker
```

7.4 Lancement du conteneur

```
docker-compose up -d
```

---

8. Vérification sur la seconde VM

```
docker ps
```

Connexion :

mysql -h IP_SGBD2 -P 3307 -u glpi -p

---

9. Conclusion

L’utilisation de Docker permet de déployer rapidement un serveur MariaDB de manière standardisée. Grâce au transfert du dossier de configuration via SCP, il est possible de reproduire facilement l’installation sur une autre machine virtuelle, garantissant ainsi une homogénéité de l’infrastructure et un gain de temps significatif lors du déploiement.

IP

GLPI1 172.26.17.2

GLPI2 172.26.17.3

HAPROXY 172.26.17.4

AD 172.26.17.5

SGBD1 172.26.17.6

SGBD2 172.26.17.7

SVR BACKUP 172.26.17.8

FAIL2BAN 172.26.17.9

SCRIPT

Machine debian 12/13 qui va stocker les sauvegardes ainsi que les scripts.

Savoir faire un CC avec mail.

<u>Éviter le mot de passe avec SSH</u> (à faire sur cette machine)

ssh-keygen
ssh-copy-id root@*IP_de_la_machine_cible*

Fichier /etc/msmrprc à modifier

```
  GNU nano 8.4                                                    msmtprc
defaults
auth           on
tls            on
tls_trust_file /etc/ssl/certs/ca-certificates.crt
logfile        /var/log/msmtp.log

account gmail
host smtp.gmail.com
port 587
from alexandrealsina11@gmai.com
user alexandrealsina11@gmail.com
password [redacted]

account default : gmail
```

Script backup_full glpi/bdd

```
#!/bin/bash
# Sauvegarde complète GLPI + toutes les bases de données + rapport email

DATE=$(date +%F)
BACKUP_DIR="/backup-glpi/backups/full_$DATE"
LOG_FILE="/backup-glpi/logs/backup.log"

GLPI_USER="root"
GLPI_IP="172.26.17.2"

DB_USER="root"
DB_IP="172.26.17.6"

MAIL_TO="alexandrealsina11@gmail.com"
```

```
HOSTNAME=$(hostname)

# Variables statut
DB_STATUS="OK"
GLPI_STATUS="OK"

# Fonction log
log() {
    echo "[$(date '+%F %T')] $1" >> $LOG_FILE
    logger "$1"
}

# Création dossier
mkdir -p $BACKUP_DIR

log "Début sauvegarde complète"

# ============================
# 1. Sauvegarde BDD
# ============================
ssh $DB_USER@$DB_IP mysqldump --all-databases > $BACKUP_DIR/all_databases.sql

if [ $? -eq 0 ]; then
    log "Toutes les BDD sauvegardées OK"
    DB_STATUS="OK"
else
    log "ERREUR sauvegarde BDD"
    DB_STATUS="ECHEC"
fi

# ============================
# 2. Sauvegarde fichiers GLPI
# ============================
mkdir -p $BACKUP_DIR/glpi

rsync -av $GLPI_USER@$GLPI_IP:/var/www/html/glpi/ $BACKUP_DIR/glpi/

if [ $? -eq 0 ]; then
    log "Fichiers GLPI OK"
    GLPI_STATUS="OK"
else
    log "ERREUR fichiers GLPI"
    GLPI_STATUS="ECHEC"
fi

log "FIN sauvegarde complète"

# ============================
```

```
# 3. Rapport EMAIL
# ============================

SUBJECT="[$HOSTNAME] Backup GLPI - $DATE"

MESSAGE="
Rapport de sauvegarde GLPI

Serveur : $HOSTNAME
Date     : $DATE

----------------------------
Base de données : $DB_STATUS
Fichiers GLPI   : $GLPI_STATUS
----------------------------

Dossier : $BACKUP_DIR
Log     : $LOG_FILE
"

echo "$MESSAGE" | mail -s "$SUBJECT" $MAIL_TO
```

Script incremential

```
#!/bin/bash
# Sauvegarde GLPI — full lundi / incr autres jours + rapport email

DATE=$(date +%F)
DAY_OF_WEEK=$(date +%u)
START_TIME=$(date +%s)

BACKUP_BASE="/backup-glpi/backups"
LOG_FILE="/backup-glpi/logs/backup.log"

GLPI_USER="root"
GLPI_IP="172.26.17.2"

MAIL_TO="alexandrealsina11@gmail.com"
HOSTNAME=$(hostname)

STATUS="OK"
TYPE=""
```

```
BACKUP_DIR=""
REF="Aucune"
SIZE="0"

log() {
    echo "[$(date '+%F %T')] $1" >> "$LOG_FILE"
    logger "$1"
}

# ─── Choix type backup ──────────────────────────────────
if [ $DAY_OF_WEEK -eq 1 ]; then
    TYPE="FULL"
    BACKUP_DIR="$BACKUP_BASE/full_$DATE"
else
    TYPE="INCR"
    BACKUP_DIR="$BACKUP_BASE/incr_$DATE"
fi

mkdir -p "$BACKUP_DIR"
log "Début sauvegarde $TYPE"

# ─── Rsync ──────────────────────────────────────────────
if [ "$TYPE" = "FULL" ]; then

    rsync -av --delete \
        $GLPI_USER@$GLPI_IP:/var/www/html/glpi \
        $BACKUP_DIR

else
    LAST_INCR=$(ls -dt $BACKUP_BASE/incr_* 2>/dev/null | grep -v $BACKUP_DIR | head
-1)
    LAST_FULL=$(ls -dt $BACKUP_BASE/full_* 2>/dev/null | head -1)

    if [ -n "$LAST_INCR" ]; then
        REF="$LAST_INCR"
    elif [ -n "$LAST_FULL" ]; then
        REF="$LAST_FULL"
    else
        log "ERREUR : aucune sauvegarde de référence trouvée"
        STATUS="ECHEC"
    fi

    log "Référence utilisée : $REF"

    if [ "$STATUS" != "ECHEC" ]; then
        rsync -av --delete --link-dest="$REF" \
            $GLPI_USER@$GLPI_IP:/var/www/html/glpi \
            $BACKUP_DIR
```

```
    fi
fi

if [ $? -eq 0 ] && [ "$STATUS" != "ECHEC" ]; then
    log "Rsync $TYPE OK"
else
    log "ERREUR rsync $TYPE"
    STATUS="ECHEC"
fi

# ─── Taille ──────────────────────────────────────────────────
if [ -d "$BACKUP_DIR" ]; then
    SIZE=$(du -sh "$BACKUP_DIR" | cut -f1)
fi

END_TIME=$(date +%s)
DURATION=$((END_TIME - START_TIME))

log "FIN sauvegarde $TYPE — taille : $SIZE"

# ============================
# 📧 RAPPORT EMAIL
# ============================

SUBJECT="[$HOSTNAME] Backup GLPI $TYPE - $DATE - $STATUS"

MESSAGE="
Rapport sauvegarde GLPI

Serveur : $HOSTNAME
Date    : $DATE

----------------------------
Type backup   : $TYPE
Statut        : $STATUS
Taille        : $SIZE
Durée         : ${DURATION}s
----------------------------

Référence     : $REF
Destination   : $BACKUP_DIR
Log           : $LOG_FILE
"

echo "$MESSAGE" | mail -s "$SUBJECT" $MAIL_TO
```

Script restauration full

```
#!/bin/bash
# Restauration GLPI — FULL ou INCR + rapport email

BACKUP_BASE="/backup-glpi/backups"
LOG_FILE="/backup-glpi/logs/restore.log"

GLPI_USER="root"
GLPI_IP="172.26.17.2"
GLPI_PATH="/var/www/html/glpi"

DB_USER="root"
DB_IP="172.26.17.6"

MAIL_TO="alexandrealsina11@gmail.com"
HOSTNAME=$(hostname)

START_TIME=$(date +%s)
DATE=$(date +%F)

STATUS="OK"
GLPI_RESTORE="OK"
BDD_RESTORE="SKIP"

log() {
    echo "[$(date '+%F %T')] $1" >> "$LOG_FILE"
    logger "$1"
}

# ─── LISTE DES SAUVEGARDES (FULL + INCR) ──────────────────────
echo "Sauvegardes disponibles :"
BACKUPS=($(ls -dt "$BACKUP_BASE"/{full,incr}_* 2>/dev/null))

if [ ${#BACKUPS[@]} -eq 0 ]; then
    echo "ERREUR : aucune sauvegarde trouvée"
    exit 1
fi

for i in ${!BACKUPS[@]}; do
    echo "$((i+1))) $(basename ${BACKUPS[$i]})"
done

read -p "Choix sauvegarde : " CHOICE
SELECTED_BACKUP=${BACKUPS[$((CHOICE-1))]}
BASENAME=$(basename "$SELECTED_BACKUP")
```

```
echo ""
echo "Sauvegarde choisie : $BASENAME"

read -p "Confirmer ? (oui/non) : " CONFIRM
if [ "$CONFIRM" != "oui" ]; then
    echo "Annulé"
    exit 0
fi

# ─── TYPE DE SAUVEGARDE ──────────────────────────────
if [[ $BASENAME == full_* ]]; then
    TYPE="FULL"
elif [[ $BASENAME == incr_* ]]; then
    TYPE="INCR"
else
    echo "Type inconnu"
    exit 1
fi

# ─── RESTORE GLPI ────────────────────────────────────
log "Début restauration GLPI ($TYPE)"

rsync -av --delete \
    $SELECTED_BACKUP/glpi/ \
    $GLPI_USER@$GLPI_IP:$GLPI_PATH

if [ $? -eq 0 ]; then
    log "GLPI restauré OK"
else
    log "ERREUR restauration GLPI"
    GLPI_RESTORE="ECHEC"
    STATUS="ECHEC"
fi

# ─── RESTORE BDD (UNIQUEMENT FULL) ───────────────────
if [ "$TYPE" == "FULL" ]; then
    log "Début restauration BDD"

    ssh $DB_USER@$DB_IP \
        "mysql -u root < $SELECTED_BACKUP/all_databases.sql"

    if [ $? -eq 0 ]; then
        log "BDD restaurée OK"
        BDD_RESTORE="OK"
    else
        log "ERREUR restauration BDD"
        BDD_RESTORE="ECHEC"
        STATUS="ECHEC"
```

```
    fi
else
    log "Restauration BDD ignorée (sauvegarde incrémentielle)"
fi

END_TIME=$(date +%s)
DURATION=$((END_TIME - START_TIME))

log "FIN restauration"

# ─── RAPPORT EMAIL ──────────────────────────────────────────
SUBJECT="[$HOSTNAME] RESTORE GLPI - $DATE - $STATUS"

MESSAGE="
RAPPORT RESTAURATION GLPI

Serveur : $HOSTNAME
Date     : $DATE

----------------------------
Type sauvegarde : $TYPE
Statut global   : $STATUS
GLPI fichiers   : $GLPI_RESTORE
Base données    : $BDD_RESTORE
Durée           : ${DURATION}s
----------------------------

Source : $BASENAME

Log : $LOG_FILE
"

echo "$MESSAGE" | mail -s "$SUBJECT" $MAIL_TO
```

Crontab

```
# Sauvegarde complète de GLPI chaque dimanche à 2h00
# Écriture dans /backup-glpi/logs/cron_full.log et dans les journaux systèmes
0 2 * * 0 /backup-glpi/scripts/backup_full.sh 2>&1 | tee -a /backup-glpi/logs/cron_full.log |
logger -t backup_full

# Sauvegarde incrémentale de GLPI du lundi au samedi à 2h00
# Écriture dans /backup-glpi/logs/cron_incr.log et dans les journaux systèmes
0 2 * * 1-6 /backup-glpi/scripts/backup_incremential.sh 2>&1 | tee -a
/backup-glpi/logs/cron_incr.log | logger -t backup_incremential
```

```
GNU nano 8.4                                        /tmp/crontab.kX1LAE/crontab *

# Sauvegarde complète de GLPI chaque dimanche à 2h00
# Écriture dans /backup-glpi/logs/cron_full.log et dans les journaux systèmes
0 2 * * 0 /backup-glpi/scripts/backup_full.sh 2>&1 | tee -a /backup-glpi/logs/cron_full.log | logger -t backup_full

# Sauvegarde incrémentale de GLPI du lundi au samedi à 2h00
# Écriture dans /backup-glpi/logs/cron_incr.log et dans les journaux systèmes
0 2 * * 1-6 /backup-glpi/scripts/backup_incremential.sh 2>&1 | tee -a /backup-glpi/logs/cron_incr.log | logger -t backup_incremential
```

```
mars 25 13:37:11 debian systemd[1]: Started session-34.scope - Session 34 of User root.
mars 25 13:59:01 debian root[9688]: BDD sauvegarde OK
mars 25 13:59:52 debian root[9695]: Fichiers GLPI OK
mars 25 14:00:14 debian root[9699]: Compresssion svg OK
mars 25 14:13:10 debian root[9750]: Sauvegarde incrémentale OK
mars 25 14:13:31 debian root[9755]: Compression OK
mars 25 14:13:34 debian root[9757]: FIN sauvegarde incrémentale
mars 25 14:16:32 debian root[9783]: Sauvegarde incrémentale OK
mars 25 14:16:53 debian root[9788]: Compression OK
mars 25 14:16:55 debian root[9790]: FIN sauvegarde incrémentale
mars 25 14:17:01 debian CRON[9792]: pam_unix(cron:session): session opened for user root(uid=0) by ro
mars 25 14:17:01 debian CRON[9794]: (root) CMD (cd / && run-parts --report /etc/cron.hourly)
mars 25 14:17:01 debian CRON[9792]: pam_unix(cron:session): session closed for user root
mars 25 14:17:09 debian root[9800]: BDD sauvegarde OK
mars 25 14:17:31 debian root[9804]: Fichiers GLPI OK
mars 25 14:17:54 debian root[9809]: Compresssion svg OK
mars 25 14:19:38 debian root[9838]: BDD sauvegarde OK
mars 25 14:19:59 debian root[9842]: Fichiers GLPI OK
mars 25 14:21:08 debian root[9860]: BDD sauvegarde OK
mars 25 14:21:30 debian root[9866]: Fichiers GLPI OK
mars 25 14:23:22 debian root[9891]: BDD sauvegarde OK
mars 25 14:23:47 debian root[9896]: Fichiers GLPI OK
mars 25 14:33:30 debian root[9922]: BDD sauvegarde OK
mars 25 14:33:52 debian root[9930]: Fichiers GLPI OK
mars 25 14:34:13 debian root[9935]: Compression OK
mars 25 14:34:15 debian root[9938]: [2026-03-25] FIN sauvegarde
mars 25 14:47:44 debian systemd[1]: Starting systemd-tmpfiles-clean.service - Cleanup of Temporary Di
mars 25 14:47:44 debian systemd-tmpfiles[9958]: /usr/lib/tmpfiles.d/legacy.conf:14: Duplicate line fo
mars 25 14:47:44 debian systemd[1]: systemd-tmpfiles-clean.service: Deactivated successfully.
mars 25 14:47:44 debian systemd[1]: Finished systemd-tmpfiles-clean.service - Cleanup of Temporary Di
mars 25 14:54:24 debian root[9989]: Sauvegarde incrémentale OK
mars 25 14:55:59 debian root[10011]: BDD sauvegarde OK
mars 25 14:56:22 debian root[10017]: Fichiers GLPI OK
mars 25 14:56:43 debian root[10023]: Compression OK
mars 25 14:56:43 debian root[10025]: [2026-03-25] FIN sauvegarde
mars 25 15:07:05 debian root[10046]: Début sauvegarde complète
mars 25 15:08:16 debian root[10056]: Début sauvegarde complète
mars 25 15:09:01 debian root[10063]: Début sauvegarde complète
mars 25 15:09:33 debian root[10068]: Début sauvegarde complète
mars 25 15:09:35 debian root[10070]: BDD sauvegardée OK
mars 25 15:09:36 debian root[10073]: ERREUR fichiers GLPI
mars 25 15:10:24 debian root[10082]: Début sauvegarde complète
mars 25 15:10:27 debian root[10084]: BDD sauvegardée OK
mars 25 15:10:28 debian root[10087]: ERREUR fichiers GLPI
mars 25 15:12:11 debian root[10100]: Début sauvegarde complète
```

Onglet 14

🧾 FICHE DESCRIPTIVE DE RÉALISATION PROFESSIONNELLE – FICHE 1

NOM et prénom du candidat :

(à compléter)

N° candidat :

(à compléter)

Contexte de la situation professionnelle

La Maison des Ligues de Lorraine (M2L) a pour mission de fournir des espaces et des services aux différentes ligues sportives régionales et aux structures hébergées.
Dans le cadre de l'évolution de son système d'information, la M2L souhaite moderniser son infrastructure réseau, renforcer la sécurité de ses services et améliorer la disponibilité de ses ressources informatiques.

Intitulé de la situation professionnelle

Mise en place et sécurisation d'une infrastructure virtualisée comprenant un environnement Active Directory redondé, services réseau (DNS/DHCP), solution GLPI, accès distant sécurisé, analyse réseau et sécurisation avancée (MFA, DNSSEC, tests d'intrusion).

Période de réalisation :

Septembre 2024 – Avril 2025

Lieu :

(ton établissement)

Modalité :

☑ En équipe
☑ Réalisations personnelles identifiées

Principales activités concernées

- A1.1.1 Analyse du cahier des charges d'un service à produire
- A1.3.4 Déploiement d'un service
- A1.4.1 Participation à un projet
- A3.2.1 Installation et configuration d'éléments d'infrastructure
- A3.2.2 Remplacement ou mise à jour d'éléments défectueux ou obsolètes
- A3.2.3 Mise à jour de la documentation technique
- A3.3.1 Administration sur site ou à distance des serveurs et réseaux
- A3.3.3 Gestion des identités et habilitations
- A4.1.9 Rédaction d'une documentation technique
- A5.1.2 Recueil d'informations sur une configuration et ses éléments

---

Conditions de réalisation

Ressources fournies :

- Hyperviseurs Proxmox
- Serveurs Windows Server
- Machines clientes Windows
- GLPI
- Wireshark
- Outils d'administration réseau
- Documentation technique

Résultats attendus :

- Mise en place d'une infrastructure virtualisée opérationnelle
- Domaine Active Directory principal et secondaire fonctionnel
- DNS/DHCP redondés
- GLPI accessible et fonctionnel
- Accès distant sécurisé
- MFA sur Proxmox
- DNSSEC activé

- Analyse réseau réalisée
- Tests de sécurité sur l’AD

Productions associées

- Documentation d’installation Proxmox / ESXi
- Procédure de création du domaine AD
- Configuration DNS / DHCP
- Documentation GLPI
- Captures Wireshark
- Procédure MFA Proxmox
- Documentation DNSSEC
- Rapport de tests AD

Modalités d’accès aux productions

(Drive / clé USB / dossier local à compléter)

Présentation de la situation professionnelle

La M2L souhaite faire évoluer son infrastructure informatique afin d’assurer une meilleure disponibilité des services, renforcer la sécurité des accès et faciliter l’administration du parc informatique.

Pour répondre à ce besoin, une infrastructure virtualisée a été mise en place via Proxmox. Un domaine Active Directory principal et secondaire a été déployé afin d’assurer la redondance des services d’authentification. Les services DNS et DHCP ont également été doublés pour garantir la continuité de service.

Une solution GLPI a été installée pour la gestion du parc et le suivi des incidents. L’accès distant aux serveurs a été sécurisé, notamment par la mise en place d’une authentification multifacteur sur Proxmox.

Enfin, des mécanismes de sécurisation avancés ont été étudiés et mis en œuvre, comme DNSSEC (RFC4033), l’analyse réseau avec Wireshark et des tests d’intrusion sur l’environnement Active Directory.

Démarche de réalisation suivie

Les besoins ont été analysés puis découpés en plusieurs tâches :

- Tâche 1 : Installation et configuration ESXi / Proxmox
- Tâche 2 : Création du domaine Active Directory
- Tâche 3 : Mise en place du DNS principal
- Tâche 4 : Mise en place du DHCP principal
- Tâche 5 : Configuration de l'accès distant sécurisé
- Tâche 6 : Installation de GLPI
- Tâche 7 : Analyse réseau avec Wireshark
- Tâche 8 : Mise en place du MFA sur Proxmox
- Tâche 9 : Déploiement d'un AD secondaire
- Tâche 10 : Mise en place DNS secondaire
- Tâche 11 : Mise en place DHCP secondaire
- Tâche 12 : Activation DNSSEC (RFC4033)
- Tâche 13 : Réalisation de tests d'intrusion sur Active Directory

---

Réalisation personnelle

J'ai participé à la mise en place de l'environnement virtualisé sous Proxmox.
J'ai installé et configuré le domaine Active Directory ainsi que les services DNS et DHCP associés.

J'ai mis en place un accès distant sécurisé aux serveurs afin de permettre leur administration à distance.

J'ai participé à l'installation et à la configuration de GLPI pour la gestion du parc informatique et des incidents.

Dans une démarche de sécurisation, j'ai configuré l'authentification multifacteur (MFA) sur Proxmox, mis en place un serveur Active Directory secondaire ainsi qu'un DNS et DHCP secondaires pour assurer la redondance.

J'ai également travaillé sur la sécurisation DNS via DNSSEC conformément à la RFC4033.

Enfin, j'ai utilisé Wireshark pour analyser les flux réseau et réalisé des tests d'intrusion sur l'Active Directory afin d'identifier d'éventuelles vulnérabilités.